

ENSA : École Nationale des Sciences
Appliquées_Kénitra

Conception et réalisation d'un système de surveillance sécurisée d'un répertoire sous Linux



Réaliser par :

- Khaliqi Salma
- Rhemimet Meryem

Encadré par :

Abderrahim Abdellaoui

Table des matières

1. Introduction :	4
2. Contexte et problématique.....	4
2.1. Importance de la surveillance sous Linux.....	4
2.2. Risques liés aux modifications non autorisées	4
2.3. Problématique du projet	5
3. Objectifs du projet.....	5
3.1. Objectif général	5
3.2. Objectifs spécifiques.....	5
4. Analyse des besoins	5
4.1. Besoins fonctionnels	5
4.2. Besoins non fonctionnels	6
5. Architecture de la solution	6
5.1. Architecture générale	6
5.2. Description des modules	7
5.3. Schéma de l'architecture	7
6. Technologies et outils utilisés	8
7. Implémentation du projet.....	9
7.1. Initialisation du système	9
7.2. Surveillance avec inotify	10
7.3. Gestion des threads	11
7.4. Journalisation des événements	11
7.5. Détection des activités suspectes	12
7.6. Gestion des signaux.....	12
7.7. Affichage des statistiques	13
8. Tests et validation	13
8.1. Compilation du projet.....	14
8.2. Vérification d'un répertoire inexistant.....	14
8.3. Lancement de la surveillance	15
8.4. Test de création d'un fichier	15
8.5. Test de modification d'un fichier	16

8.6. Test de suppression d'un fichier	16
8.7. Test de renommage	17
8.8. Test des sous-répertoires	17
8.9. Test des fichiers cachés	18
8.10. Test des extensions interdites	18
8.11. Test des fichiers exécutables	18
8.12. Test des suppressions répétées	19
8.13. Test des modifications rapides	19
8.14. Test des statistiques	21
8.15. Test de l'arrêt propre	21
8.16. Vérification du fichier de journalisation	22
9. Difficultés rencontrées	24
9.1. Duplication des événements inotify	24
9.2. Synchronisation entre threads	24
9.3. Gestion des renommages de fichiers	24
9.4. Gestion de l'arrêt propre du système	25
10. Résultats obtenus	25
10.1. Fonctionnement global	25
10.2. Performances	25
10.3. Stabilité	26
10.4. Conformité au cahier des charges	26
11. Conclusion et perspectives	26
11.1. Conclusion du projet	26
11.2. Perspectives d'amélioration	27
12. Conclusion générale	28

1. Introduction :

Dans un système Linux, certains répertoires peuvent contenir des fichiers sensibles ou importants nécessitant une surveillance continue afin de détecter toute activité anormale ou non autorisée. Dans ce contexte, ce projet consiste à développer une application en langage C capable de surveiller un répertoire en temps réel à l'aide du mécanisme inotify du noyau Linux.

L'application SecureBox permet de détecter les créations, suppressions, modifications et renommages de fichiers, de journaliser les événements dans un fichier log, de générer des alertes en cas d'activité suspecte et d'afficher des statistiques sur l'exécution du système. Ce projet met en pratique plusieurs concepts de programmation système sous Linux tels que les appels système, les threads, les signaux et la gestion des fichiers.

2. Contexte et problématique

2.1. Importance de la surveillance sous Linux

Sous Linux, certains répertoires peuvent contenir des fichiers importants ou sensibles comme des documents, des fichiers de configuration, des journaux système ou des codes sources. La surveillance de ces répertoires permet de détecter rapidement les changements effectués sur les fichiers afin d'assurer la sécurité et l'intégrité des données.

Dans les systèmes informatiques, il est important de pouvoir suivre les activités réalisées sur les fichiers, notamment les créations, modifications, suppressions ou renommages. Une surveillance en temps réel permet également d'identifier des comportements suspects et de réagir rapidement en cas de problème.

2.2. Risques liés aux modifications non autorisées

Les modifications non autorisées peuvent provoquer plusieurs problèmes tels que :

- la suppression accidentelle de fichiers importants ;
- la modification de données sensibles ;
- l'ajout de fichiers suspects ou malveillants ;
- l'exécution de scripts dangereux ;
- la perte de l'intégrité des données.

Sans mécanisme de surveillance, ces activités peuvent passer inaperçues et compromettre le bon fonctionnement du système.

2.3. Problématique du projet

Le principal problème traité dans ce projet est la conception d'un système capable de surveiller un répertoire sous Linux en temps réel, de détecter les événements importants sur les fichiers et de signaler les activités suspectes tout en garantissant un fonctionnement fiable et sécurisé.

Le système doit également enregistrer les événements dans un fichier journal, produire des alertes et assurer un arrêt propre de l'application sans perte de données ni ressources ouvertes.

3. Objectifs du projet

3.1. Objectif général

L'objectif principal de ce projet est de développer une application en langage C sous Linux permettant de surveiller un répertoire en temps réel, de détecter les événements liés aux fichiers et de signaler les activités suspectes tout en assurant une bonne gestion des ressources système.

3.2. Objectifs spécifiques

Ce projet vise à :

- surveiller un répertoire fourni par l'utilisateur ;
- détecter les créations, suppressions, modifications et renommages de fichiers ;
- enregistrer les événements dans un fichier journal ;
- générer des alertes en cas d'activité suspecte ;
- afficher des statistiques sur les événements détectés ;
- gérer les signaux système pour permettre un arrêt propre ;
- mettre en pratique les notions de programmation système sous Linux telles que les threads, les appels système et le mécanisme inotify.

4. Analyse des besoins

4.1. Besoins fonctionnels

Le système doit assurer plusieurs fonctionnalités essentielles afin de garantir une surveillance efficace du répertoire cible.

Fonctionnalité	Description
Création	Détecter la création de nouveaux fichiers et sous-répertoires.
Suppression	Détecter la suppression des fichiers et dossiers.
Modification	Détecter les modifications effectuées sur les fichiers.
Renommage	Détecter les opérations de renommage ou déplacement.
Journalisation	Enregistrer les événements dans un fichier log.
Alertes	Générer des alertes lors d'activités suspectes.
Statistiques	Afficher les statistiques des événements détectés.
Arrêt propre	Libérer correctement les ressources lors de la fermeture.

4.2. Besoins non fonctionnels

Besoin	Description
Performance	Détecter les événements rapidement avec une faible consommation de ressources.
Fiabilité	Garantir un fonctionnement stable sans blocage du système.
Sécurité	Éviter l'utilisation de commandes dangereuses et protéger les données surveillées.
Maintenabilité	Organiser le code en modules clairs et réutilisables.
Portabilité	Assurer la compatibilité avec les systèmes Linux utilisant GCC.
Lisibilité	Produire des logs clairs et compréhensibles.

5. Architecture de la solution

5.1. Architecture générale

L'application SecureBox adopte une architecture modulaire basée sur plusieurs composants spécialisés. Chaque module possède une responsabilité précise afin d'assurer une meilleure organisation du code, une maintenance plus simple et une meilleure fiabilité du système.

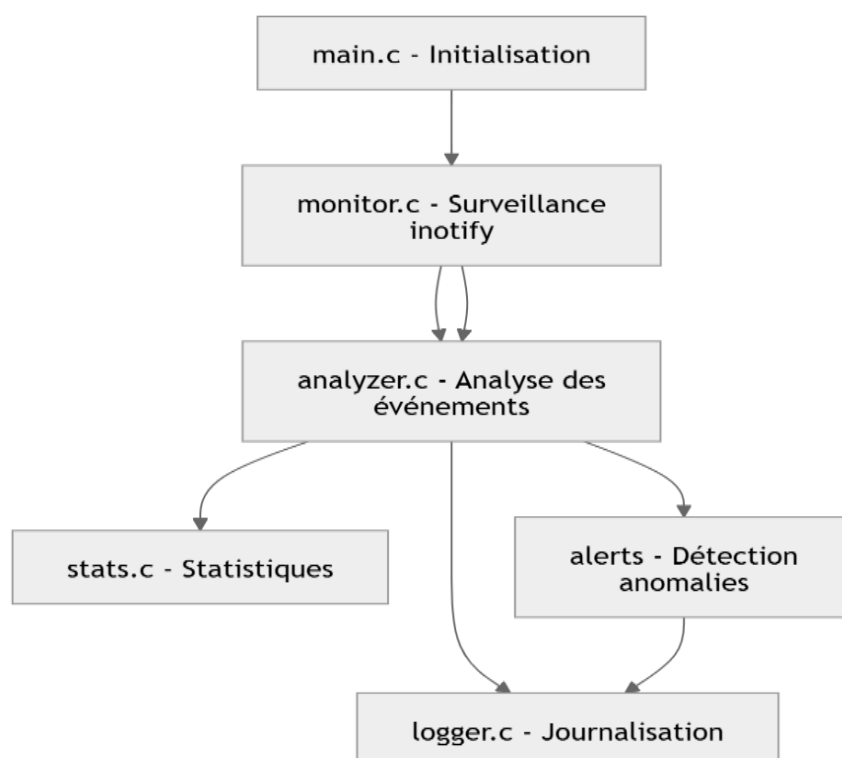
Le programme principal initialise les ressources nécessaires, démarre les différents modules puis lance la surveillance du répertoire cible. Les événements détectés sont

ensuite analysés, enregistrés dans un journal et utilisés pour mettre à jour les statistiques ou générer des alertes.

5.2. Description des modules

Module	Rôle
Module de surveillance	Utilise inotify pour détecter en temps réel les événements sur les fichiers et répertoires.
Module d'analyse	Analyse les événements détectés afin d'identifier les activités suspectes.
Module de journalisation	Enregistre les événements et alertes dans le fichier log.
Module de statistiques	Maintient les compteurs des événements et affiche les statistiques du système.
Module de signaux	Gère les signaux système comme SIGINT et SIGTERM pour assurer un arrêt propre.
Module principal	Coordonne l'ensemble des modules et initialise les ressources.

5.3. Schéma de l'architecture



Le schéma ci-dessus représente l'architecture globale du système **SecureBox**. Il met en évidence l'organisation modulaire de l'application ainsi que les échanges entre ses différents composants. Le système est conçu selon une approche modulaire afin de séparer clairement les responsabilités et faciliter la maintenance et l'évolution du code.

L'architecture est composée de six modules principaux :

Le module d'initialisation est le point d'entrée du système. Il est responsable du lancement de l'application, de la lecture des arguments (chemin du répertoire à surveiller), de la vérification de son existence ainsi que de l'initialisation des différentes ressources nécessaires (fichiers log, threads, structures de données, etc.).

Le module de surveillance assure la détection en temps réel des événements sur le répertoire cible à l'aide de l'API inotify de Linux. Il capte les événements liés aux fichiers et sous-répertoires (création, suppression, modification, renommage).

Le module d'analyse reçoit les événements détectés et les interprète afin de déterminer leur nature exacte. Il applique également les règles de détection des comportements suspects (modifications rapides, fichiers cachés, extensions interdites, etc.).

Le module de détection des anomalies est chargé d'identifier les comportements considérés comme suspects. Il se base sur les résultats de l'analyse et les règles de sécurité définies pour générer des alertes.

Le module de journalisation enregistre tous les événements détectés dans un fichier log structuré. Il garantit la traçabilité des actions effectuées sur le système avec un format contenant la date, le type d'événement et le chemin concerné.

Le module de statistiques maintient des compteurs globaux sur les événements traités (créations, suppressions, modifications, renommages et alertes). Il permet également de fournir un résumé global de l'activité du système lors de l'exécution ou à la fin du programme.

Les interactions entre ces modules sont principalement séquentielles : les événements sont d'abord détectés par le module de surveillance, puis analysés, ensuite journalisés, et enfin comptabilisés par le module de statistiques. Le module de détection des anomalies intervient entre l'analyse et la journalisation pour filtrer les comportements suspects.

Cette architecture garantit une séparation claire des responsabilités, améliore la lisibilité du code et facilite les tests ainsi que la maintenance du système SecureBox.

6. Technologies et outils utilisés

Le développement du projet SecureBox repose sur un ensemble de technologies et d'outils du système Linux permettant la programmation système et la gestion des événements en temps réel.

Langage C

Le langage C a été choisi pour sa proximité avec le système d'exploitation Linux, sa performance et sa capacité à manipuler directement les appels système.

Linux

Le projet est développé et exécuté sous Linux, car ce système fournit les mécanismes nécessaires à la surveillance des fichiers (inotify), la gestion des processus et des signaux.

GCC (GNU Compiler Collection)

GCC est utilisé pour compiler le code source du projet et générer l'exécutable. Il offre une bonne compatibilité avec les bibliothèques système.

Makefile

Le Makefile permet d'automatiser la compilation du projet, en facilitant la gestion des fichiers sources et des dépendances.

pthread (POSIX Threads)

Les threads sont utilisés pour permettre l'exécution concurrente des modules (surveillance, analyse, journalisation), améliorant ainsi la réactivité du système.

inotify

inotify est une API du noyau Linux utilisée pour détecter en temps réel les événements sur le système de fichiers (création, suppression, modification, renommage).

Signaux POSIX

Les signaux sont utilisés pour gérer l'arrêt propre du programme et permettre une terminaison contrôlée du système sans perte de données.

7. Implémentation du projet

7.1. Initialisation du système

L'initialisation du système est réalisée dans le fichier `main.c`. Cette étape prépare tous les modules nécessaires avant le démarrage de la surveillance.

Le programme commence par vérifier les arguments passés en ligne de commande afin de s'assurer qu'un répertoire cible a bien été fourni. Ensuite, il vérifie l'existence et les droits d'accès du répertoire grâce à la fonction `check_directory()` définie dans `utils.c`.

Après cette vérification, le programme initialise les différents modules :

- installation des gestionnaires de signaux (`signals_init`) ;

- initialisation du système de journalisation (`logger_init`) ;
- initialisation des statistiques (`stats_init`) ;
- initialisation du module d'analyse (`analyzer_init`) ;
- initialisation du module de surveillance (`monitor_init`).

Le fichier `main.c` crée ensuite trois threads principaux :

- thread de surveillance ;
- thread d'analyse ;
- thread de journalisation.

Enfin, le programme affiche une bannière d'informations contenant le répertoire surveillé, le fichier log et le PID du processus.

7.2. Surveillance avec inotify

La surveillance du système de fichiers est implémentée dans le fichier `monitor.c` à l'aide de l'API Linux `inotify`.

Le module initialise `inotify` avec la fonction :

```
g_inotify_fd = inotify_init();
```

Puis il ajoute une surveillance sur le répertoire cible grâce à :

```
inotify_add_watch()
```

Le système détecte plusieurs types d'événements :

- création de fichiers ;
- suppression ;
- modification ;
- renommage ;
- changement d'attributs.

Les événements sont récupérés à travers un buffer contenant les structures `inotify_event`. Chaque événement est ensuite converti en structure `FsEvent` avant d'être envoyé au module d'analyse via :

```
analyzer_push(&evt);
```

Le module gère également certains problèmes liés à `inotify`, notamment :

- suppression des doublons de création/modification ;
- association des événements `IN_MOVED_FROM` et `IN_MOVED_TO` ;
- filtrage du fichier `securebox.log`.

7.3. Gestion des threads

Le projet utilise la bibliothèque `pthread` afin d'exécuter plusieurs tâches simultanément.

Trois threads principaux sont créés :

Thread	Rôle
<code>monitor_thread</code>	Surveillance des événements du système de fichiers
<code>analyser_thread</code>	Analyse des événements et détection des anomalies
<code>logger_thread</code>	Écriture des événements dans le fichier log

Les threads communiquent entre eux grâce à des files d'attente protégées par :

- `pthread_mutex_t`
- `pthread_cond_t`

Cette architecture permet :

- une meilleure réactivité ;
- une exécution parallèle ;
- une séparation claire des responsabilités.

7.4. Journalisation des événements

Le module de journalisation est implémenté dans `logger.c`.

Son rôle est d'enregistrer tous les événements détectés dans le fichier :

logs/securebox.log

Chaque entrée contient :

- la date ;
- l'heure ;
- le niveau du message ;
- le type d'événement ;
- le chemin du fichier concerné.

Exemple :

[2026-05-08 00:13:22] [INFO] CREATE_FILE : ./testdir/test.sh

Le logger utilise une file d'attente afin d'éviter les accès concurrents au fichier log.

Le thread `logger_thread` récupère les messages et les écrit :

- dans le fichier log ;

- sur la console en temps réel.

7.5. Détection des activités suspectes

Le module d'analyse est implémenté dans `analyzer.c`.

Il reçoit les événements depuis le module de surveillance et applique plusieurs règles de sécurité afin de détecter des comportements suspects.

Les principales règles implémentées sont :

Règle	Description
HIDDEN_FILE	Détection des fichiers cachés
FORBIDDEN_EXT	Détection des extensions interdites
EXEC_CREATED	Détection des fichiers exécutables
Rafale de suppressions	Plusieurs suppressions en peu de temps
Modifications rapides	Modifications répétées sur un même fichier

Exemple d'alerte :

```
[ALERTE] FORBIDDEN_EXT : ./testdir/test.sh
```

Lorsqu'une anomalie est détectée :

- une alerte est générée ;
- l'événement est journalisé ;
- les statistiques sont mises à jour.

7.6. Gestion des signaux

La gestion des signaux est réalisée dans `signals.c`.

Le programme intercepte plusieurs signaux POSIX :

Signal	Rôle
SIGINT	Arrêt avec Ctrl+C
SIGTERM	Arrêt propre du programme
SIGUSR1	Affichage des statistiques

Les gestionnaires sont installés avec :

```
sigaction()
```

Lorsqu'un signal d'arrêt est reçu :

- un message est affiché ;
- les threads sont arrêtés proprement ;
- les ressources sont libérées ;

- les statistiques finales sont affichées.

Cette approche permet d'éviter :

- la corruption du fichier log ;
- les threads bloqués ;
- les ressources non libérées.

7.7. Affichage des statistiques

Le module `stats.c` maintient des statistiques globales sur les événements détectés.

Les compteurs mis à jour sont :

- nombre total d'événements ;
- créations ;
- suppressions ;
- modifications ;
- renommages ;
- alertes générées.

Les statistiques sont protégées par un mutex afin d'éviter les problèmes de concurrence entre threads.

Lors de l'arrêt du programme ou après réception du signal `SIGUSR1`, les statistiques sont affichées sous la forme suivante :

```
=====
STATISTIQUES SECUREBOX
=====
Total événements : 10
Créations      : 3
Suppressions   : 2
Modifications  : 4
Renommages     : 1
Alertes générées : 2
=====
```

Les statistiques sont également enregistrées dans le fichier log afin de conserver un historique complet de l'activité du système.

8. Tests et validation

Afin de vérifier le bon fonctionnement du système SecureBox, plusieurs scénarios de test ont été réalisés. Les tests ont permis de valider la détection des événements système, la génération des alertes, la journalisation ainsi que la gestion correcte des statistiques et des signaux.

8.1. Compilation du projet

Avant la compilation, le répertoire du projet contient uniquement les fichiers sources, les fichiers d'en-tête, le fichier Makefile ainsi que les dossiers du projet.

Structure du projet avant compilation

```
salma@salma-VMware-Virtual-Platform:~/securebox$ ls
Makefile README.md analyzer.c analyzer.h config.h logger.c logger.h logs main.c monitor.c
monitor.h signals.c signals.h stats.c stats.h testdir utils.c utils.h
salma@salma-VMware-Virtual-Platform:~/securebox$
```

La compilation du projet est ensuite réalisée à l'aide de la commande suivante :

```
make
```

Résultat de la compilation

```
salma@salma-VMware-Virtual-Platform:~/securebox$ make
gcc -Wall -Wextra -std=gnu11 -pthread -D_GNU_SOURCE -c -o main.o main.c
gcc -Wall -Wextra -std=gnu11 -pthread -D_GNU_SOURCE -c -o monitor.o monitor.c
gcc -Wall -Wextra -std=gnu11 -pthread -D_GNU_SOURCE -c -o analyzer.o analyzer.c
gcc -Wall -Wextra -std=gnu11 -pthread -D_GNU_SOURCE -c -o logger.o logger.c
gcc -Wall -Wextra -std=gnu11 -pthread -D_GNU_SOURCE -c -o stats.o stats.c
gcc -Wall -Wextra -std=gnu11 -pthread -D_GNU_SOURCE -c -o signals.o signals.c
gcc -Wall -Wextra -std=gnu11 -pthread -D_GNU_SOURCE -c -o utils.o utils.c
gcc -Wall -Wextra -std=gnu11 -pthread -D_GNU_SOURCE -o securebox main.o monitor.o an
alyzer.o logger.o stats.o signals.o utils.o -pthread
Compilation réussie -> ./securebox
salma@salma-VMware-Virtual-Platform:~/securebox$
```

Après la compilation, les fichiers objets .o ainsi que l'exécutable securebox sont générés automatiquement.

Structure du projet après compilation

```
salma@salma-VMware-Virtual-Platform:~/securebox$ ls
Makefile analyzer.h logger.c logs monitor.c securebox signals.o stats.o utils.h
README.md analyzer.o logger.h main.c monitor.h signals.c stats.c testdir utils.o
analyzer.c config.h logger.o main.o monitor.o signals.h stats.h utils.c
salma@salma-VMware-Virtual-Platform:~/securebox$
```

→ La compilation du projet s'est effectuée correctement grâce au fichier Makefile, qui automatise la génération des fichiers objets et de l'exécutable principal.

8.2. Vérification d'un répertoire inexistant

Commande

```
./securebox ./inexistent
```

Résultat

```
salma@salma-VMware-Virtual-Platform:~/securebox$ ./securebox ./inexistant
[ERREUR] Impossible d'accéder à './inexistant' : No such file or directory
salma@salma-VMware-Virtual-Platform:~/securebox$
```

→ Le programme vérifie correctement l'existence du répertoire avant le démarrage de la surveillance.

8.3. Lancement de la surveillance

Commande

```
./securebox ./testdir
```

Résultat

```
salma@salma-VMware-Virtual-Platform:~/securebox$ ./securebox ./testdir
[SecureBox] Surveillance démarrée sur : ./testdir
```

SecureBox – Surveillance Linux
ENSA – Programmation Système GI S8

```
Répertoire surveillé : ./testdir
Fichier log           : logs/securebox.log
Arrêt propre          : Ctrl+C ou kill -SIGTERM <pid>
Statistiques          : kill -SIGUSR1 <pid>
PID                   : 7216

[SecureBox] Surveillance active. En attente d'événements...

[2026-05-09 19:21:18] [START ] Surveillance démarrée sur : ./testdir
[2026-05-09 19:21:18] [START ] SecureBox démarré avec succès.
```

Le système démarre correctement et affiche :

- le répertoire surveillé ;
- le fichier de journalisation ;
- le PID du processus ;
- les commandes de gestion des signaux.

→ La surveillance du répertoire est correctement initialisée avec l'API inotify.

8.4. Test de création d'un fichier

Commande

```
touch testdir/rapport.txt
```

Résultat

```
Compilation réussie → ./sec
salma@salma-VMware-Virtual-Platform: ~/securebox
[ERREUR] Impossible d'accéder
salma@salma-VMware-Virtual-Platform: ~/securebox$ touch testdir/rapport.txt
[SecureBox] Surveillance démarrée sur : ./testdir
salma@salma-VMware-Virtual-Platform: ~/securebox$

SecureBox - Surveillance
ENSA - Programmation

Répertoire surveillé : ./
Fichier log : log
Arrêt propre : Ctrl+C
Statistiques : kill
PID : 72

[SecureBox] Surveillance démarrée sur : ./testdir
[2026-05-09 19:21:18] [START] Surveillance démarrée sur : ./testdir
[2026-05-09 19:21:18] [START] SecureBox démarré avec succès.
[2026-05-09 19:22:11] [INFO] CREATE_FILE : ./testdir/rapport.txt
```

→ Le système détecte correctement la création d'un nouveau fichier.

8.5. Test de modification d'un fichier

Commande

```
echo "bonjour" >> testdir/rapport.txt
```

Résultat

```
salma@salma-VMware-Virtual-Platform: ~/securebox
[SecureBox] Surveillance démarrée sur : ./testdir
salma@salma-VMware-Virtual-Platform: ~/securebox$ touch testdir/rapport.txt
salma@salma-VMware-Virtual-Platform: ~/securebox$ echo "bonjour" >> testdir/rapport.txt
salma@salma-VMware-Virtual-Platform: ~/securebox$

SecureBox - Surveillance
ENSA - Programmation

Répertoire surveillé : ./
Fichier log : log
Arrêt propre : Ctrl+C
Statistiques : kill
PID : 72

[SecureBox] Surveillance démarrée sur : ./testdir
[2026-05-09 19:21:18] [START] Surveillance démarrée sur : ./testdir
[2026-05-09 19:21:18] [START] SecureBox démarré avec succès.
[2026-05-09 19:22:11] [INFO] CREATE_FILE : ./testdir/rapport.txt
[2026-05-09 19:23:54] [INFO] MODIFY_FILE : ./testdir/rapport.txt
```

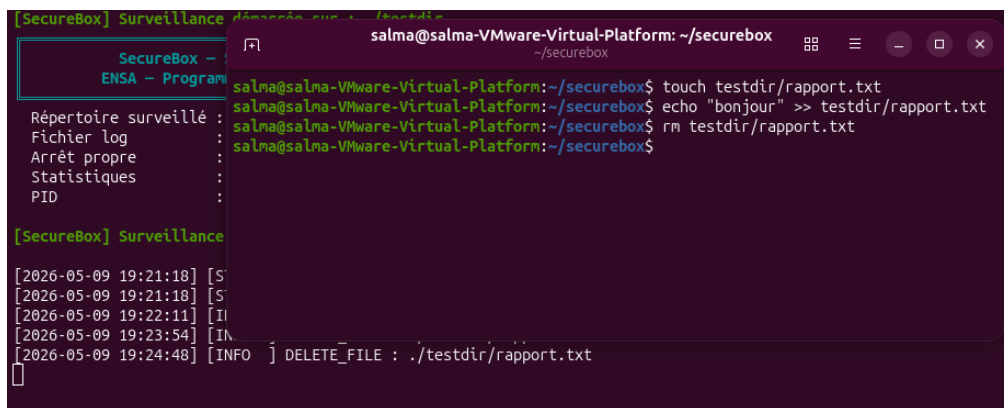
→ Les modifications d'un fichier existant sont correctement détectées.

8.6. Test de suppression d'un fichier

Commande

```
rm testdir/rapport.txt
```

Résultat



```
[SecureBox] Surveillance
Répertoire surveillé : /
Fichier log : /var/log/securebox.log
Arrêt propre :
Statistiques :
PID :

[SecureBox] Surveillance
[2026-05-09 19:21:18] [S]
[2026-05-09 19:21:18] [S]
[2026-05-09 19:22:11] [I]
[2026-05-09 19:23:54] [I]
[2026-05-09 19:24:48] [INFO ] DELETE_FILE : ./testdir/rapport.txt
```

→ Le système détecte correctement la suppression des fichiers.

8.7. Test de renommage

Commande

```
mv testdir/ancien.txt testdir/nouveau.txt
```

Résultat



```
Arrêt propre
Statistiques
PID

[SecureBox] Surveill
[2026-05-09 19:21:18]
[2026-05-09 19:21:18]
[2026-05-09 19:22:11]
[2026-05-09 19:23:54]
[2026-05-09 19:24:48]
[2026-05-09 19:25:42] [INFO ] CREATE_FILE : ./testdir/ancien.txt
[2026-05-09 19:25:48] [INFO ] RENAME_FILE : ./testdir/ancien.txt -> ./testdir/nouveau.txt
```

→ Les opérations de renommage sont correctement détectées grâce aux événements `IN_MOVED_FROM` et `IN_MOVED_TO`.

8.8. Test des sous-répertoires

Commande

```
mkdir testdir/sousrep
rmdir testdir/sousrep
```

Résultat

```
[securebox] sur votre
[2026-05-09 19:21:18] salma@salma-VMware-Virtual-Platform:~/securebox$ mkdir testdir/sousrep
[2026-05-09 19:21:18] salma@salma-VMware-Virtual-Platform:~/securebox$ rmdir testdir/sousrep
[2026-05-09 19:22:11] salma@salma-VMware-Virtual-Platform:~/securebox$
[2026-05-09 19:23:54]
[2026-05-09 19:24:48]
[2026-05-09 19:25:42]
[2026-05-09 19:25:48]
[2026-05-09 19:27:26] [INFO ] CREATE_DIR : ./testdir/sousrep
[2026-05-09 19:27:36] [INFO ] DELETE_DIR : ./testdir/sousrep
█
```

→ Le système prend également en charge les opérations sur les répertoires.

8.9. Test des fichiers cachés

Commande

```
touch testdir/.secret
```

Résultat

```
[2026-05-09 19:22:11] salma@salma-VMware-Virtual-Platform:~/securebox$ touch testdir/.secret
[2026-05-09 19:23:54] salma@salma-VMware-Virtual-Platform:~/securebox$
[2026-05-09 19:24:48]
[2026-05-09 19:25:42]
[2026-05-09 19:25:48]
[2026-05-09 19:27:26]
[2026-05-09 19:27:36] [INFO ] CREATE_FILE : ./testdir/.secret
[2026-05-09 19:28:17] [INFO ] HIDDEN_FILE : ./testdir/.secret
[2026-05-09 19:28:17] [ALERTE] HIDDEN_FILE : ./testdir/.secret
█
```

→ Le module d'analyse détecte correctement les fichiers cachés considérés comme suspects.

8.10. Test des extensions interdites

Commande

```
touch testdir/virus.exe
touch testdir/malware.sh
```

Résultat

```
[2026-05-09 19:25:42] salma@salma-VMware-Virtual-Platform:~/securebox$ touch testdir/virus.exe
[2026-05-09 19:25:48] salma@salma-VMware-Virtual-Platform:~/securebox$ touch testdir/malware.sh
[2026-05-09 19:27:26] salma@salma-VMware-Virtual-Platform:~/securebox$ █
[2026-05-09 19:28:17]
[2026-05-09 19:28:17] [INFO ] CREATE_FILE : ./testdir/virus.exe
[2026-05-09 19:29:01] [ALERTE] FORBIDDEN_EXT : ./testdir/virus.exe
[2026-05-09 19:29:13] [INFO ] CREATE_FILE : ./testdir/malware.sh
[2026-05-09 19:29:13] [ALERTE] FORBIDDEN_EXT : ./testdir/malware.sh
█
```

→ Le système identifie correctement les extensions définies dans la liste noire.

8.11. Test des fichiers exécutables

Commande

```
touch testdir/script.sh
chmod +x testdir/script.sh
```

Résultat

```
[2026-05-09 19:29:01] [INFO] salma@salma-VMware-Virtual-Platform:~/securebox$ touch testdir/script.sh
[2026-05-09 19:29:01] [INFO] salma@salma-VMware-Virtual-Platform:~/securebox$ chmod +x testdir/script.sh
[2026-05-09 19:29:13] [INFO] salma@salma-VMware-Virtual-Platform:~/securebox$ 
[2026-05-09 19:30:04] [INFO] CREATE_FILE : ./testdir/script.sh
[2026-05-09 19:30:04] [ALERTE] FORBIDDEN_EXT : ./testdir/script.sh
[2026-05-09 19:30:14] [ALERTE] EXEC_CHMOD : ./testdir/script.sh
```

→ Le programme détecte correctement l'ajout des permissions d'exécution.

8.12. Test des suppressions répétées

Commande

```
touch testdir/f{1..10}
rm testdir/f*
```

Résultat

```
[2026-05-09 19:30:04] [INFO] salma@salma-VMware-Virtual-Platform:~/securebox$ touch testdir/f{1..10}
[2026-05-09 19:30:04] [ALERTE] salma@salma-VMware-Virtual-Platform:~/securebox$ rm testdir/f*
[2026-05-09 19:30:14] [ALERTE] salma@salma-VMware-Virtual-Platform:~/securebox$ 
[2026-05-09 19:31:34] [INFO] CREATE_FILE : ./testdir/f1
[2026-05-09 19:31:34] [INFO] CREATE_FILE : ./testdir/f2
[2026-05-09 19:31:34] [INFO] CREATE_FILE : ./testdir/f3
[2026-05-09 19:31:34] [INFO] CREATE_FILE : ./testdir/f4
[2026-05-09 19:31:34] [INFO] CREATE_FILE : ./testdir/f5
[2026-05-09 19:31:34] [INFO] CREATE_FILE : ./testdir/f6
[2026-05-09 19:31:34] [INFO] CREATE_FILE : ./testdir/f7
[2026-05-09 19:31:34] [INFO] CREATE_FILE : ./testdir/f8
[2026-05-09 19:31:34] [INFO] CREATE_FILE : ./testdir/f9
[2026-05-09 19:31:34] [INFO] CREATE_FILE : ./testdir/f10
[2026-05-09 19:31:48] [INFO] DELETE_FILE : ./testdir/f1
[2026-05-09 19:31:48] [INFO] DELETE_FILE : ./testdir/f10
[2026-05-09 19:31:48] [INFO] DELETE_FILE : ./testdir/f2
[2026-05-09 19:31:48] [INFO] DELETE_FILE : ./testdir/f3
[2026-05-09 19:31:48] [INFO] DELETE_FILE : ./testdir/f4
[2026-05-09 19:31:48] [INFO] DELETE_FILE : ./testdir/f5
[2026-05-09 19:31:48] [ALERTE] Rafale de suppressions detectee (>5 en 10s)
[2026-05-09 19:31:48] [INFO] DELETE_FILE : ./testdir/f6
[2026-05-09 19:31:48] [INFO] DELETE_FILE : ./testdir/f7
[2026-05-09 19:31:48] [INFO] DELETE_FILE : ./testdir/f8
[2026-05-09 19:31:48] [INFO] DELETE_FILE : ./testdir/f9
```

→ Le système détecte correctement une activité anormale de suppressions multiples dans une courte période.

8.13. Test des modifications rapides

Ce test a été réalisé en deux étapes afin de distinguer la création initiale d'un fichier des modifications répétées sur un fichier déjà existant.

Étape 1: Création du fichier

Commande

```
for i in {1..10}; do echo "x" >> testdir/file1; done
```

Résultat

```
salma@salma-VMware-Virtual-Platform:~/securebox$ for i in {1..10}; do echo "x" >> testdir/file1; done
salma@salma-VMware-Virtual-Platform:~/securebox$
[2026-05-09 19:34:59] [INFO] CREATE_FILE : ./testdir/file1
[2026-05-09 19:34:59] [INFO] MODIFY_FILE : ./testdir/file1
```

Explication

Lors de cette première commande, le fichier `file1` n'existait pas encore dans le répertoire surveillé.

Le système détecte donc uniquement un événement de création (`CREATE_FILE`).

Même si plusieurs écritures sont effectuées dans la boucle, elles interviennent dans le même contexte de création initiale du fichier.

De plus, le mécanisme de surveillance basé sur `inotify` peut regrouper ces opérations, ce qui explique qu'un seul événement soit affiché.

→ Le système identifie correctement la création initiale du fichier sans générer de faux événements de modification.

Étape 2 : Modifications répétées rapides

Commande

```
touch testdir/f{1..10}
rm testdir/f*
```

Résultat

```
salma@salma-VMware-Virtual-Platform:~/securebox$ for i in {1..5}; do echo "x" >> testdir/file1; sleep 0.2; done
salma@salma-VMware-Virtual-Platform:~/securebox$
[2026-05-09 19:35:43] [INFO] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:35:43] [INFO] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:35:43] [INFO] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:35:43] [INFO] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:35:43] [ALERTE] Modifications rapides sur : ./testdir/file1 (>3 en 5s)
[2026-05-09 19:35:44] [INFO] MODIFY_FILE : ./testdir/file1
```

Explication

Dans cette seconde étape, le fichier existe déjà dans le système de fichiers.

Chaque opération d'écriture est donc considérée comme une modification réelle (`MODIFY_FILE`).

Comme plusieurs modifications sont détectées dans un intervalle de temps très court (plus de 3 modifications en moins de 5 secondes), le module d'analyse interprète ce comportement comme suspect et déclenche automatiquement une alerte de sécurité.

→ Le système détecte correctement les modifications répétées rapides et applique les règles de détection d'anomalies définies dans le module d'analyse.

- Ce test met en évidence la différence entre une opération de création initiale et une série de modifications successives, ainsi que la capacité du système à filtrer et analyser les événements en temps réel.

8.14. Test des statistiques

Commande

```
kill -SIGUSR1 $(pgrep securebox)
```

Résultat

```
[2] salma@salma-VMware-Virtual-Platform:~/securebox$ kill -SIGUSR1 $(pgrep securebox)
[2] salma@salma-VMware-Virtual-Platform:~/securebox$

=====
STATISTIQUES SECUREBOX
=====
Durée de fonctionnement : 0h 15m 19s
Total événements       : 46
Créations               : 19
Suppressions           : 13
Modifications          : 13
Renommages              : 1
Alertes générées       : 8
=====
```

Affichage des statistiques du système :

- nombre total d'événements ;
- créations ;
- suppressions ;
- modifications ;
- renommages ;
- alertes générées.

→ Le système affiche correctement les statistiques en temps réel sans arrêter la surveillance.

8.15. Test de l'arrêt propre

Action :

Appui sur:

```
Ctrl+C
```

Résultat

```
[2026-05-09 19:33:06] [INFO ] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:34:42] [INFO ] DELETE_FILE : ./testdir/file1
[2026-05-09 19:34:59] [INFO ] CREATE_FILE : ./testdir/file1
[2026-05-09 19:34:59] [INFO ] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:35:43] [INFO ] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:35:43] [INFO ] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:35:43] [INFO ] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:35:43] [INFO ] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:35:43] [ALERTE] Modifications rapides sur : ./testdir/file1 (>3 en 5s)
[2026-05-09 19:35:44] [INFO ] MODIFY_FILE : ./testdir/file1

=====
STATISTIQUES SECUREBOX
=====
Durée de fonctionnement : 0h 15m 19s
Total événements       : 46
Créations              : 19
Suppressions           : 13
Modifications          : 13
Renommages             : 1
Alertes générées       : 8
=====

^C
[SecureBox] Signal d'arrêt reçu. Fermeture en cours...

[SecureBox] Arrêt en cours...

=====
STATISTIQUES SECUREBOX
=====
Durée de fonctionnement : 0h 16m 0s
Total événements       : 46
Créations              : 19
Suppressions           : 13
Modifications          : 13
Renommages             : 1
Alertes générées       : 8
=====

[2026-05-09 19:37:18] [STOP ] SecureBox arrêté proprement.
[SecureBox] Ressources libérées. Au revoir.
salma@salma-VMware-Virtual-Platform:~/securebox$
```

Le système :

- arrête les threads ;
- libère les ressources ;
- ferme les fichiers ;
- affiche les statistiques finales.

→ Le mécanisme de gestion des signaux fonctionne correctement.

8.16. Vérification du fichier de journalisation

Le contenu du fichier logs/securebox.log:

```

salma@salma-VMware-Virtual-Platform:~/securebox$ cat logs/securebox.log
[2026-05-09 19:21:18] [START ] Surveillance démarrée sur : ./testdir
[2026-05-09 19:21:18] [START ] SecureBox démarré avec succès.
[2026-05-09 19:22:11] [INFO ] CREATE_FILE : ./testdir/rapport.txt
[2026-05-09 19:23:54] [INFO ] MODIFY_FILE : ./testdir/rapport.txt
[2026-05-09 19:24:48] [INFO ] DELETE_FILE : ./testdir/rapport.txt
[2026-05-09 19:25:42] [INFO ] CREATE_FILE : ./testdir/ancien.txt
[2026-05-09 19:25:48] [INFO ] RENAME_FILE : ./testdir/ancien.txt -> ./testdir/nouveau.txt
[2026-05-09 19:27:26] [INFO ] CREATE_DIR : ./testdir/sousrep
[2026-05-09 19:27:36] [INFO ] DELETE_DIR : ./testdir/sousrep
[2026-05-09 19:28:17] [INFO ] CREATE_FILE : ./testdir/.secret
[2026-05-09 19:28:17] [ALERTE] HIDDEN_FILE : ./testdir/.secret
[2026-05-09 19:29:01] [INFO ] CREATE_FILE : ./testdir/virus.exe
[2026-05-09 19:29:01] [ALERTE] FORBIDDEN_EXT : ./testdir/virus.exe
[2026-05-09 19:29:13] [INFO ] CREATE_FILE : ./testdir/malware.sh
[2026-05-09 19:29:13] [ALERTE] FORBIDDEN_EXT : ./testdir/malware.sh
[2026-05-09 19:30:04] [INFO ] CREATE_FILE : ./testdir/script.sh
[2026-05-09 19:30:04] [ALERTE] FORBIDDEN_EXT : ./testdir/script.sh
[2026-05-09 19:30:14] [ALERTE] EXEC_CHMOD : ./testdir/script.sh
[2026-05-09 19:31:34] [INFO ] CREATE_FILE : ./testdir/f1
[2026-05-09 19:31:34] [INFO ] CREATE_FILE : ./testdir/f2
[2026-05-09 19:31:34] [INFO ] CREATE_FILE : ./testdir/f3
[2026-05-09 19:31:34] [INFO ] CREATE_FILE : ./testdir/f4
[2026-05-09 19:31:34] [INFO ] CREATE_FILE : ./testdir/f5
[2026-05-09 19:31:34] [INFO ] CREATE_FILE : ./testdir/f6
[2026-05-09 19:31:34] [INFO ] CREATE_FILE : ./testdir/f7
[2026-05-09 19:31:34] [INFO ] CREATE_FILE : ./testdir/f8
[2026-05-09 19:31:34] [INFO ] CREATE_FILE : ./testdir/f9
[2026-05-09 19:31:34] [INFO ] CREATE_FILE : ./testdir/f10
[2026-05-09 19:31:48] [INFO ] DELETE_FILE : ./testdir/f1
[2026-05-09 19:31:48] [INFO ] DELETE_FILE : ./testdir/f10
[2026-05-09 19:31:48] [INFO ] DELETE_FILE : ./testdir/f2
[2026-05-09 19:31:48] [INFO ] DELETE_FILE : ./testdir/f3
[2026-05-09 19:31:48] [INFO ] DELETE_FILE : ./testdir/f4
[2026-05-09 19:31:48] [INFO ] DELETE_FILE : ./testdir/f5
[2026-05-09 19:31:48] [ALERTE] Rafale de suppressions detectee (>5 en 10s)
[2026-05-09 19:31:48] [INFO ] DELETE_FILE : ./testdir/f6
[2026-05-09 19:31:48] [INFO ] DELETE_FILE : ./testdir/f7
[2026-05-09 19:31:48] [INFO ] DELETE_FILE : ./testdir/f8

```

```

[2026-05-09 19:31:48] [INFO ] DELETE_FILE : ./testdir/f4
[2026-05-09 19:31:48] [INFO ] DELETE_FILE : ./testdir/f5
[2026-05-09 19:31:48] [ALERTE] Rafale de suppressions detectee (>5 en 10s)
[2026-05-09 19:31:48] [INFO ] DELETE_FILE : ./testdir/f6
[2026-05-09 19:31:48] [INFO ] DELETE_FILE : ./testdir/f7
[2026-05-09 19:31:48] [INFO ] DELETE_FILE : ./testdir/f8
[2026-05-09 19:31:48] [INFO ] DELETE_FILE : ./testdir/f9
[2026-05-09 19:32:35] [INFO ] CREATE_FILE : ./testdir/file1
[2026-05-09 19:33:05] [INFO ] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:33:05] [INFO ] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:33:06] [INFO ] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:33:06] [INFO ] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:33:06] [ALERTE] Modifications rapides sur : ./testdir/file1 (>3 en 5s)
[2026-05-09 19:33:06] [INFO ] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:34:42] [INFO ] DELETE_FILE : ./testdir/file1
[2026-05-09 19:34:59] [INFO ] CREATE_FILE : ./testdir/file1
[2026-05-09 19:34:59] [INFO ] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:35:43] [INFO ] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:35:43] [INFO ] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:35:43] [INFO ] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:35:43] [INFO ] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:35:43] [ALERTE] Modifications rapides sur : ./testdir/file1 (>3 en 5s)
[2026-05-09 19:35:44] [INFO ] MODIFY_FILE : ./testdir/file1
[2026-05-09 19:36:37] [STATS ] =====
[2026-05-09 19:36:37] [STATS ] Durée de fonctionnement : 0h 15m 19s
[2026-05-09 19:36:37] [STATS ] Total événements : 46
[2026-05-09 19:36:37] [STATS ] Créations : 19
[2026-05-09 19:36:37] [STATS ] Suppressions : 13
[2026-05-09 19:36:37] [STATS ] Modifications : 13
[2026-05-09 19:36:37] [STATS ] Renommages : 1
[2026-05-09 19:36:37] [STATS ] Alertes générées : 8
[2026-05-09 19:36:37] [STATS ] =====
[2026-05-09 19:37:18] [STATS ] =====
[2026-05-09 19:37:18] [STATS ] Durée de fonctionnement : 0h 16m 0s
[2026-05-09 19:37:18] [STATS ] Total événements : 46
[2026-05-09 19:37:18] [STATS ] Créations : 19
[2026-05-09 19:37:18] [STATS ] Suppressions : 13
[2026-05-09 19:37:18] [STATS ] Modifications : 13
[2026-05-09 19:37:18] [STATS ] Renommages : 1
[2026-05-09 19:37:18] [STATS ] Alertes générées : 8
[2026-05-09 19:37:18] [STATS ] =====
[2026-05-09 19:37:18] [STOP ] SecureBox arrêté proprement.
salma@salma-VMware-Virtual-Platform:~/securebox$

```

contient l'ensemble des événements affichés dans la console.

→ La journalisation persistante fonctionne correctement et garantit la traçabilité des événements détectés.

9. Difficultés rencontrées

Durant le développement du projet SecureBox, plusieurs difficultés techniques ont été rencontrées, notamment liées à la gestion des événements système et à la programmation concurrente.

9.1. Duplication des événements inotify

L'une des premières difficultés concernait la duplication des événements générés par inotify.

Certains événements apparaissaient plusieurs fois (par exemple lors des écritures dans un fichier), ce qui pouvait fausser les statistiques.

Pour résoudre ce problème, des mécanismes de filtrage et de regroupement des événements ont été ajoutés, notamment l'utilisation de `IN_CLOSE_WRITE` et des contrôles temporels afin de réduire le bruit des événements.

9.2. Synchronisation entre threads

Le projet utilise plusieurs threads (surveillance, analyse, journalisation).

Une difficulté importante était la **synchronisation des accès aux files d'événements**.

Sans protection, des conditions de concurrence (race conditions) pouvaient apparaître.

Ce problème a été résolu en utilisant :

- des mutex (`pthread_mutex`)
- des variables de condition (`pthread_cond`)
- une file circulaire sécurisée pour les événements

9.3. Gestion des renommages de fichiers

La détection correcte des opérations de renommage était complexe, car inotify génère deux événements distincts :

- `IN_MOVED_FROM`
- `IN_MOVED_TO`

Il était nécessaire de les associer correctement à l'aide d'un identifiant `cookie`.

Un système de mise en attente temporaire a été implémenté afin de reconstruire correctement l'événement de renommage.

9.4. Gestion de l'arrêt propre du système

Une autre difficulté concernait l'arrêt propre du programme à l'aide des signaux (SIGINT, SIGTERM).

Il fallait s'assurer que :

- les threads se terminent correctement
- les ressources système soient libérées
- le programme ne se bloque pas lors de l'arrêt

Cette difficulté a été résolue en utilisant :

- des handlers de signaux avec `sigaction`
- un flag global d'arrêt
- la fermeture contrôlée des threads avec `pthread_join`

→ Ces difficultés ont permis de mieux comprendre la programmation système sous Linux, notamment la gestion des événements en temps réel et la synchronisation entre processus et threads.

10. Résultats obtenus

Cette section présente une synthèse des résultats obtenus après l'implémentation et les tests du système SecureBox. L'objectif est d'évaluer le comportement global de l'application par rapport aux exigences initiales.

10.1. Fonctionnement global

Le système SecureBox fonctionne de manière conforme aux objectifs définis dans le cahier des charges.

Il permet la surveillance en temps réel d'un répertoire donné et détecte correctement les événements liés aux fichiers et dossiers (création, suppression, modification et renommage).

L'ensemble des modules (surveillance, analyse, journalisation, statistiques et gestion des signaux) fonctionne de manière coordonnée grâce à une architecture multi-thread sécurisée.

10.2. Performances

Les performances du système sont satisfaisantes pour un outil de surveillance en temps réel :

- Détection quasi immédiate des événements grâce à `inotify`

- Traitement des événements en file d'attente sans blocage majeur
- Utilisation optimisée des threads pour répartir les tâches (surveillance, analyse, log)

Le système reste réactif même lors de fortes activités dans le répertoire surveillé.

10.3. Stabilité

Le système présente une bonne stabilité lors des tests :

- Aucun crash observé durant les scénarios de test
- Gestion correcte des accès concurrents grâce aux mutex
- Arrêt propre du programme via les signaux SIGINT et SIGTERM
- Libération correcte des ressources (threads, fichiers, inotify)

Ces éléments garantissent un fonctionnement fiable sur de longues durées.

10.4. Conformité au cahier des charges

Le projet respecte globalement les objectifs fixés :

- Surveillance des événements fichiers en temps réel
- Détection des activités suspectes (fichiers cachés, extensions interdites, modifications rapides, etc.)
- Journalisation des événements dans un fichier log
- Affichage et calcul de statistiques d'activité
- Gestion des signaux pour arrêt propre et affichage des statistiques

Ainsi, le système répond aux exigences fonctionnelles et non fonctionnelles définies au début du projet.

→ Les résultats obtenus montrent que SecureBox est un système fonctionnel, stable et efficace pour la surveillance de répertoires sous Linux, intégrant correctement les mécanismes de détection et d'analyse des événements système.

11. Conclusion et perspectives

11.1. Conclusion du projet

Le projet **SecureBox** a permis de concevoir et d'implémenter un système de surveillance de fichiers sous Linux basé sur l'API inotify. L'application est capable de détecter en temps réel les événements du système de fichiers (création, suppression, modification et renommage), de les analyser et de générer des alertes en cas de comportements suspects.

L'architecture modulaire (surveillance, analyse, journalisation, statistiques et gestion des signaux) a permis de structurer le code de manière claire et maintenable.

L'utilisation des threads et des mécanismes de synchronisation a également permis d'assurer un traitement fluide et concurrent des événements.

Globalement, le projet répond aux objectifs fixés et démontre une bonne maîtrise des concepts de programmation système sous Linux.

11.2. Perspectives d'amélioration

Malgré les fonctionnalités déjà implémentées, plusieurs améliorations peuvent être envisagées pour enrichir le système :

- **Surveillance récursive**

Actuellement, la surveillance est limitée à un seul répertoire.

Une amélioration importante serait d'ajouter la **surveillance récursive**, permettant de monitorer automatiquement tous les sous-répertoires.

Cela nécessiterait la gestion dynamique des `inotify watch` pour chaque nouveau dossier créé.

- **Export des données en CSV**

Le système pourrait être enrichi par une fonctionnalité d'export des événements et statistiques dans un fichier **CSV**.

Cela permettrait :

- une analyse externe des logs
- une exploitation dans Excel ou Python
- un suivi historique plus structuré

- **Interface graphique**

Une évolution possible serait de développer une **interface graphique (GUI)** permettant de visualiser en temps réel :

- les événements détectés
- les alertes de sécurité
- les statistiques du système

Cela améliorerait fortement l'ergonomie du projet pour l'utilisateur final.

- **Quarantaine automatique**

Une fonctionnalité de sécurité avancée pourrait être ajoutée : la **mise en quarantaine automatique** des fichiers suspects.

Par exemple :

- déplacement des fichiers dangereux vers un dossier sécurisé
- blocage automatique des fichiers exécutables non autorisés
- journalisation de l'action de quarantaine

→ Ces améliorations permettraient de transformer SecureBox en un outil de sécurité plus complet et proche des solutions professionnelles de monitoring système.

12. Conclusion générale

Le projet **SecureBox** a permis de concevoir et développer un système complet de surveillance de répertoires sous Linux en utilisant le langage C et les mécanismes système avancés tels que inotify, les threads POSIX et les signaux.

L'objectif principal consistait à détecter en temps réel les événements liés au système de fichiers (création, modification, suppression et renommage) tout en assurant leur analyse et leur journalisation. Grâce à une architecture modulaire composée de plusieurs composants (monitoring, analyse, logging, statistiques et gestion des signaux), le système fonctionne de manière cohérente et efficace.

Les tests réalisés montrent que SecureBox est capable de :

- surveiller un répertoire en temps réel,
- détecter correctement les événements du système de fichiers,
- identifier des comportements suspects et générer des alertes,
- enregistrer les événements dans un fichier de log,
- fournir des statistiques détaillées sur l'activité du système,
- s'arrêter proprement via les signaux système.

Cependant, certaines limites subsistent, notamment l'absence de surveillance récursive et d'interface graphique, qui pourraient améliorer l'ergonomie et les capacités du système.

En conclusion, ce projet a été une expérience enrichissante permettant de renforcer les compétences en programmation système sous Linux, en gestion de la concurrence et en conception d'architectures logicielles robustes. Il constitue une base solide pouvant être étendue vers des solutions de sécurité plus avancées.